

Uniwersytet WSB Merito

Wydział Studiów Inżynierskich

Kierunek: Informatyka

[miejsce na logo uczelni — zgodnie ze wzorem WSB]

PROJEKT ZALICZENIOWY

Wdrożenie aplikacji mobilnej VetMed w oparciu o infrastrukturę chmurową Google Cloud Platform

Przedmiot: [nazwa przedmiotu]

Prowadzący: [tytuł, imię i nazwisko prowadzącego]

Autorzy (grupa projektowa):

Robert [nazwisko], nr albumu: [.....]

[imię i nazwisko 2, nr albumu]

[imię i nazwisko 3, nr albumu]

[imię i nazwisko 4, nr albumu]

[Miasto] 2026

Spis treści

1. Krótki opis aplikacji i jej funkcjonalności	3
1.1. Struktura rozwiązania	3
1.2. Najważniejsze funkcjonalności aplikacji mobilnej.....	3
2. Architektura aplikacji w Google Cloud Platform	5
2.1. Diagram architektury	5
2.2. Komponenty infrastruktury.....	5
2.3. Przepływ żądania i bezpieczeństwo.....	6
2.4. Koszty infrastruktury	6
3. Technologie wykorzystane do wdrożenia aplikacji.....	8
3.1. Konteneryzacja — Docker.....	8
3.2. Baza danych i migracje.....	8
3.3. Konfiguracja i sekrety.....	8
3.4. Pozostałe narzędzia	8
4. CI/CD — proces ciągłej integracji i wdrażania	9
4.1. Narzędzia w pipeline.....	9
4.2. Przebieg wdrożenia API	9
4.3. Wdrożenie aplikacji mobilnej	10
4.4. Środowiska i częstotliwość wydań	10
5. Wnioski.....	11
5.1. Uzasadnienie założeń projektowych.....	11
5.2. Uzasadnienie wyboru technologii.....	11
5.3. Czego nauczyliśmy się podczas projektu	11
5.4. Możliwości rozwoju i poprawa efektywności wdrażania.....	12
5.5. Plany dalszego rozwoju aplikacji.....	12

1. Krótki opis aplikacji i jej funkcjonalności

VetMed to system informatyczny wspierający obsługę przychodni weterynaryjnej od strony właściciela zwierzęcia. Sercem rozwiązania jest aplikacja mobilna na system Android, która komunikuje się przez internet z interfejsem REST API wdrożonym w chmurze Google Cloud Platform. Dane przechowywane są w zarządzanej bazie PostgreSQL (Cloud SQL). Projekt został zrealizowany w całości w ekosystemie .NET 9 i języku C#.

1.1. Struktura rozwiązania

Moduł	Technologia	Rola
VetMed.App	.NET MAUI Blazor Hybrid (Android)	aplikacja mobilna dla właściciela zwierząt — wersja 1.1.0 (build 2)
VetMed.Api	ASP.NET Core 9 (REST + JWT)	interfejs API wdrażany w chmurze (Cloud Run)
VetMed.Clinic	Blazor Server	panel administracyjny kliniki (uruchamiany lokalnie, planowany do wdrożenia)
VetMed.Infrastructure	EF Core 9 + Npgsql	warstwa dostępu do danych, migracje, seeder
VetMed.Shared	biblioteka klas (.NET 9)	wspólne modele i DTO (rekordy)
VetMed.Tests	xUnit	testy jednostkowe

1.2. Najważniejsze funkcjonalności aplikacji mobilnej

- ☐ rejestracja i logowanie użytkownika (uwierzytelnianie tokenem JWT),
- ☐ zarządzanie zwierzętami: dodawanie ze zdjęciem, edycja profilu, archiwizacja, automatyczne wyliczanie wieku z daty urodzenia,
- ☐ karta pacjenta z zakładkami: opis, lista wizyt, szczepienia,
- ☐ rezerwacja wizyty w krokach: wybór zwierzęcia, rodzaju wizyty (szczepienie / kontrola / zabieg / nagły przypadek), lekarza, daty w kalendarzu i wolnej godziny (zajęte sloty są blokowane),
- ☐ zmiana terminu i odwoływanie zarezerwowanych wizyt,
- ☐ historia leczenia w formie osi czasu z filtrowaniem po zwierzęciu,

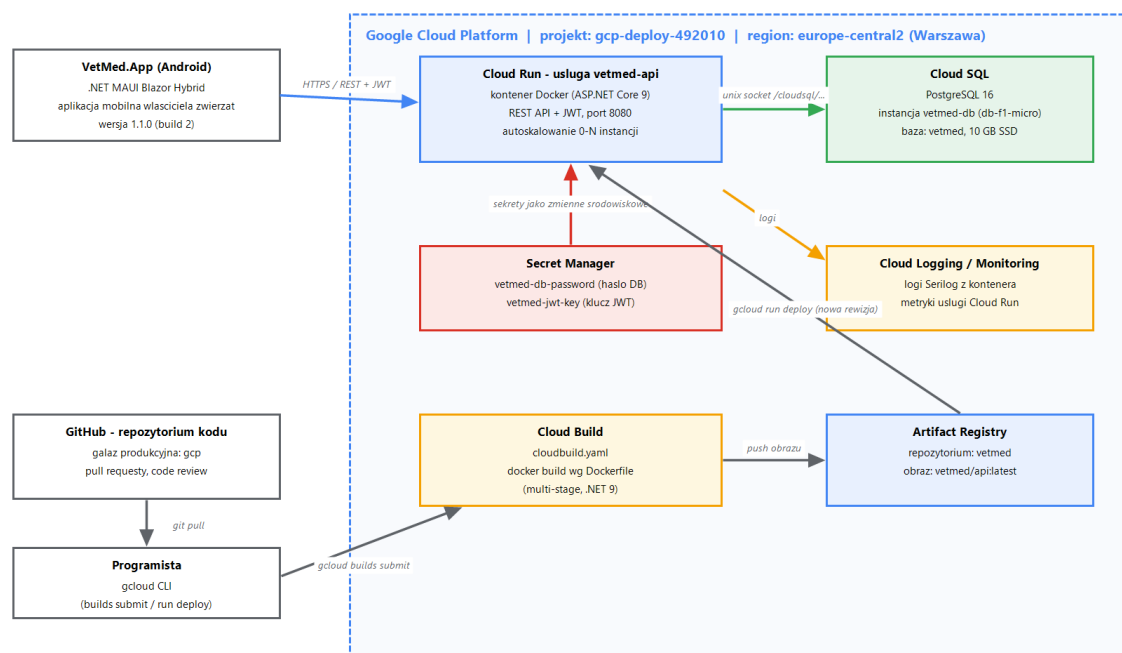
- ☐ powiadomienia pogrupowane czasowo (dziś / ten tydzień / starsze),
- ☐ wyszukiwarka (zwierzęta, historia wizyt), profil użytkownika, ustawienia z trybem ciemnym i jasnym,
- ☐ informacje o przychodni z nawigacją do Map Google i natywnym połączeniem telefonicznym.

2. Architektura aplikacji w Google Cloud Platform

Architektura opiera się na usługach zarządzanych (serverless i PaaS), dzięki czemu projekt nie wymaga utrzymywania własnych maszyn wirtualnych. Wszystkie zasoby chmurowe znajdują się w projekcie gcp-deploy-492010, w regionie europe-central2 (Warszawa) — najbliższym geograficznie użytkownikom aplikacji, co minimalizuje opóźnienia i zapewnia przetwarzanie danych na terenie Unii Europejskiej.

2.1. Diagram architektury

Poniższy diagram (przygotowany w narzędziu draw.io — edytowalny plik VetMed-Architektura-GCP.drawio znajduje się w repozytorium projektu) przedstawia komponenty systemu oraz przepływy danych i procesu wdrożenia.



Rys. 1. Architektura systemu VetMed w Google Cloud Platform (opracowanie własne, draw.io)

2.2. Komponenty infrastruktury

Usługa GCP	Konfiguracja w projekcie	Zadanie
Cloud Run	usługa vetmed-api, kontener Docker, port 8080, autoskalowanie 0–N instancji	hostowanie REST API; publiczny adres HTTPS dla aplikacji mobilnej
Cloud SQL	PostgreSQL 16, instancja vetmed-db (db-f1-micro, 10 GB SSD), baza vetmed	zarządzana baza danych; brak publicznego dostępu z internetu

Usługa GCP	Konfiguracja w projekcie	Zadanie
Artifact Registry	repozytorium vetmed, obraz europe-central2-docker.pkg.dev/.../vetmed/api:latest	przechowywanie i wersjonowanie obrazów Docker
Cloud Build	plik cloudbuild.yaml w repozytorium	budowanie obrazu Docker w chmurze (bez lokalnego Dockera)
Secret Manager	sekrety: vetmed-db-password, vetmed-jwt-key	bezpieczne przechowywanie hasła do bazy i klucza JWT
Cloud Logging	logi Serilog ze standardowego wyjścia kontenera	centralne logowanie i diagnostyka usługi
IAM	konto usługi Cloud Run z rolami Cloud SQL Client i Secret Manager Secret Accessor	kontrola dostępu zgodna z zasadą najmniejszych uprawnień

2.3. Przepływ żądania i bezpieczeństwo

Aplikacja mobilna komunikuje się wyłącznie z API poprzez HTTPS — telefon nigdy nie łączy się bezpośrednio z bazą danych. Cloud Run łączy się z Cloud SQL przez dedykowany łącznik (gniazdo unix /cloudsql/<nazwa instancji>), więc baza nie posiada publicznego adresu IP. Hasło do bazy oraz klucz podpisujący tokeny JWT nie znajdują się w repozytorium kodu (pola w appsettings.json są puste) — są wstrzykiwane do kontenera jako zmienne środowiskowe z Secret Manager (ConnectionStrings__DefaultConnection, Jwt__Key). Dostęp użytkownika do danych chroniony jest tokenem JWT o ważności 120 minut.

2.4. Koszty infrastruktury

Usługa	W okresie kredytu 300 USD	Po wyczerpaniu kredytu
Cloud SQL (db-f1-micro)	0 USD	ok. 8–10 USD / mies.
Cloud Run (mały ruch)	0 USD	zwykle 0 USD (darmowy limit + skalowanie do zera)
Artifact Registry, Secret Manager, Cloud Build	marginalne	marginalne (groszowe kwoty)

Projekt świadomie wykorzystuje najtańsze warianty usług — wystarczające na potrzeby prototypu i obrony projektu, a jednocześnie identyczne architektonicznie z konfiguracją produkcyjną, którą można skalować bez zmian w kodzie.

3. Technologie wykorzystane do wdrożenia aplikacji

3.1. Konteneryzacja — Docker

API jest pakowane do obrazu Docker zdefiniowanego w pliku Dockerfile w trybie multi-stage: etap build używa obrazu mcr.microsoft.com/dotnet/sdk:9.0 (przywrócenie pakietów NuGet i publikacja w konfiguracji Release), a etap finalny — lekkiego obrazu uruchomieniowego mcr.microsoft.com/dotnet/aspnet:9.0. Dzięki temu finalny obraz nie zawiera SDK ani kodu źródłowego. Kontener nasłuchuje na porcie 8080, a numer portu odczytywany jest ze zmiennej środowiskowej PORT przekazywanej przez Cloud Run:

```
var port = Environment.GetEnvironmentVariable("PORT") ?? "5100";
app.Run($"http://0.0.0.0:{port}");
```

3.2. Baza danych i migracje

Warstwa danych korzysta z Entity Framework Core 9 ze sterownikiem Npgsql. Schemat bazy wersjonowany jest migracjami EF Core, które wykonują się automatycznie przy starcie aplikacji (db.Database.MigrateAsync()), po czym uruchamiany jest seeder wypełniający bazę danymi startowymi (m.in. lekarze i rodzaje wizyt). Dzięki temu świeżo wdrożona rewizja samodzielnie aktualizuje schemat produkcyjnej bazy — nie jest potrzebny ręczny krok migracji.

3.3. Konfiguracja i sekrety

Aplikacja realizuje zasadę konfiguracji przez środowisko (podejście 12-factor): parametry połączenia z bazą i ustawienia JWT pochodzą ze zmiennych środowiskowych ustawianych podczas wdrożenia usługi Cloud Run, a wartości wrażliwe — z Secret Manager. Lokalna konfiguracja deweloperska (appsettings.json) nie zawiera żadnych haseł.

3.4. Pozostałe narzędzia

- ❑ gcloud CLI — tworzenie zasobów, budowanie obrazów (gcloud builds submit) i wdrażanie (gcloud run deploy),
- ❑ docker-compose — lokalna baza PostgreSQL 16 do pracy deweloperskiej,
- ❑ Serilog — strukturalne logowanie do konsoli, automatycznie zbierane przez Cloud Logging,
- ❑ Scalar / OpenAPI — interaktywna dokumentacja API w środowisku deweloperskim,
- ❑ dotnet publish dla platformy net9.0-android — budowanie pakietu APK aplikacji mobilnej.

4. CI/CD — proces ciągłej integracji i wdrażania

4.1. Narzędzia w pipeline

Narzędzie	Rola w procesie
Git + GitHub	kontrola wersji; gałęzie funkcyjne (feature/*), pull requesty z przeglądem kodu, gałąź wdrożeniowa gcp
Cloud Build	budowanie obrazu Docker w chmurze według pliku cloudbuild.yaml
Artifact Registry	rejestr obrazów — przechowuje kolejne wersje obrazu api
Cloud Run (rewizje)	każde wdrożenie tworzy nową, niemodyfikowalną rewizję usługi; możliwy natychmiastowy rollback do poprzedniej
gcloud CLI	uruchamianie kroków pipeline (builds submit, run deploy)

4.2. Przebieg wdrożenia API

1. Programista rozwija funkcjonalność na gałęzi feature/* i otwiera pull request; po przeglądzie kodu zmiany trafiają do gałęzi wdrożeniowej gcp.
2. Polecenie `gcloud builds submit` wysyła źródła do Cloud Build, który wykonuje docker build zgodnie z Dockerfile (multi-stage, .NET 9).
3. Zbudowany obraz jest publikowany w Artifact Registry jako `europe-central2-docker.pkg.dev/gcp-deploy-492010/vetmed/api:latest`.
4. Polecenie `gcloud run deploy vetmed-api` tworzy nową rewizję usługi Cloud Run — z podpiętą instancją Cloud SQL oraz sekretami z Secret Manager jako zmiennymi środowiskowymi.
5. Cloud Run przełącza ruch na nową rewizję; w razie problemów możliwy jest natychmiastowy powrót (rollback) do rewizji poprzedniej.
6. Przy starcie kontenera EF Core wykonuje migracje bazy danych i seeding — wdrożenie nie wymaga żadnych ręcznych kroków po stronie bazy.

Polecenia używane w procesie:

```
gcloud builds submit # build obrazu wg cloudbuild.yaml
gcloud run deploy vetmed-api \
  --image europe-central2-docker.pkg.dev/gcp-deploy-492010/vetmed/api:latest \
  --add-cloudsql-instances <projekt>:europe-central2:vetmed-db \
  --set-secrets Jwt_Key=vetmed-jwt-key:latest ...
```

4.3. Wdrożenie aplikacji mobilnej

1. Zmiana adresu API nie jest potrzebna — wersja Release ma wkompirowany produkcyjny adres Cloud Run, a wersja Debug łączy się z lokalnym API (emulator Android: 10.0.2.2).
2. Budowanie pakietu: `dotnet publish VetMed.App -f net9.0-android -c Release`.
3. Dystrybucja pakietu APK na urządzenia testowe (sideload); docelowo publikacja w Google Play.

4.4. Środowiska i częstotliwość wydań

Środowisko	Infrastruktura	Przeznaczenie i częstotliwość
Development (lokalne)	PostgreSQL 16 w Dockerze (docker-compose), API na localhost:5100, emulator Androida	codzienna praca deweloperska; każda zmiana kodu
Produkcja (GCP)	Cloud Run + Cloud SQL + Secret Manager, region europe-central2	wydania API po zakończeniu pakietu funkcji — średnio 1–2 razy w tygodniu
Aplikacja mobilna	APK budowany lokalnie, wersjonowanie semantyczne (obecnie 1.1.0, build 2)	wydania rzadsze, skorelowane ze stabilnymi wersjami API

Dzięki rozdzieleniu API i klienta mobilnego wydania serwera mogą odbywać się często i bez udziału użytkowników — usługa Cloud Run podmienia rewizję bez przerwy w działaniu, a aplikacja na telefonie nadal korzysta z tego samego, stabilnego adresu HTTPS.

5. Wnioski

5.1. Uzasadnienie założeń projektowych

Przyjęliśmy klasyczny trójwarstwowy podział: aplikacja mobilna nie ma żadnego dostępu do bazy — całość logiki i autoryzacji przechodzi przez API. Upraszcza to bezpieczeństwo (jeden punkt wejścia, JWT) i pozwala w przyszłości podłączyć kolejnych klientów (panel kliniki) bez zmian w infrastrukturze. Region europe-central2 wybraliśmy ze względu na minimalne opóźnienia dla użytkowników w Polsce i przetwarzanie danych w UE. Trzecim założeniem była minimalizacja kosztów: usługi serverless skalujące się do zera oraz najmniejsza instancja bazy w pełni pokrywają potrzeby prototypu w ramach kredytu 300 USD dla nowych kont GCP.

5.2. Uzasadnienie wyboru technologii

- ☐ jeden język w całym projekcie — C#/ .NET 9 dla aplikacji mobilnej, API i warstwy danych ogranicza koszt utrzymania i pozwala współdzielić modele (VetMed.Shared) między klientem a serwerem,
- ☐ .NET MAUI Blazor Hybrid — komponenty UI pisane raz działają na Androidzie i Windows; zespół wykorzystuje umiejętności webowe (Blazor) do budowy aplikacji natywnej,
- ☐ Cloud Run zamiast maszyn wirtualnych (Compute Engine) czy Kubernetes (GKE) — zero administracji serwerami, płatność wyłącznie za faktyczne żądania, automatyczny HTTPS i skalowanie; przy ruchu prototypu koszt bliski zeru,
- ☐ Cloud SQL zamiast własnego PostgreSQL na VM — automatyczne kopie zapasowe, aktualizacje i łącznik eliminujący publiczną ekspozycję bazy,
- ☐ Cloud Build + Artifact Registry — budowanie obrazów w chmurze bez wymogu posiadania Dockera na maszynie deweloperskiej, spójne z resztą ekosystemu GCP,
- ☐ Secret Manager — sekrety poza repozytorium kodu, z kontrolą dostępu IAM i wersjonowaniem.

5.3. Czego nauczyliśmy się podczas projektu

- ☐ konteneryzacji aplikacji .NET (multi-stage build, minimalizacja obrazu, kontrakt portu przez zmienną PORT),
- ☐ praktycznej pracy z usługami GCP: Cloud Run, Cloud SQL, Cloud Build, Artifact Registry, Secret Manager i IAM,

- ❑ bezpiecznego łączenia usług (łącznik Cloud SQL przez gniazdo unix, role konta usługi, sekrety jako zmienne środowiskowe),
- ❑ konfiguracji aplikacji zgodnie z podejściem 12-factor i automatyzacji migracji bazy przy wdrożeniu,
- ❑ modelu rewizji Cloud Run — wdrożeń bez przerwy w działaniu i szybkiego rollbacku,
- ❑ szacowania i kontroli kosztów chmury (darmowe limity, skalowanie do zera, dobór rozmiaru instancji bazy).

5.4. Możliwości rozwoju i poprawa efektywności wdrażania

Przy większych nakładach i większej grupie odbiorców wprowadzilibyśmy następujące zmiany:

- ❑ pełna automatyzacja pipeline — trigger Cloud Build podpięty do GitHub: każdy merge do gałęzi gcp automatycznie buduje obraz, uruchamia testy (xUnit, testy integracyjne z Testcontainers) i wdraża nową rewizję bez ręcznych poleceń gcloud,
- ❑ środowisko staging — druga usługa Cloud Run i osobna baza do testów akceptacyjnych przed wydaniem produkcyjnym,
- ❑ infrastruktura jako kod (Terraform) — odtwarzalność całego środowiska z repozytorium i przeglądy zmian infrastruktury w pull requestach,
- ❑ wdrożenia kanarkowe — stopniowe przełączanie ruchu między rewizjami Cloud Run (np. 10% / 90%) ograniczające ryzyko regresji,
- ❑ monitoring i alerty — Cloud Monitoring z testami dostępności (uptime checks) i powiadomieniami o błędach; budżety i alerty kosztowe,
- ❑ skalowanie bazy — większa instancja Cloud SQL z trybem wysokiej dostępności i replikami do odczytu; cache Memorystore (Redis) dla najczęstszych zapytań,
- ❑ własna domena z Cloud Armor (WAF, limity żądań) przed publicznym API,
- ❑ dystrybucja mobilna przez Google Play — podpisany pakiet AAB budowany w pipeline, kanały testowe (internal/closed testing) i aktualizacje OTA,
- ❑ wdrożenie panelu kliniki (VetMed.Clinic) jako drugiej usługi Cloud Run korzystającej z tego samego API i bazy.

5.5. Plany dalszego rozwoju aplikacji

- ❑ publikacja aplikacji w sklepie Google Play oraz wydanie wersji na Windows,

- ☐ powiadomienia push (Firebase Cloud Messaging) o zbliżających się wizytach i szczepieniach,
- ☐ płatności online za wizyty oraz dokumenty PDF (wyniki badań, zalecenia) przechowywane w Cloud Storage,
- ☐ uruchomienie panelu kliniki w chmurze i synchronizacja kalendarzy lekarzy w czasie rzeczywistym,
- ☐ rozszerzenie o konta wielu przychodni (model multi-tenant) po zdobyciu pierwszych użytkowników.